Universidad EAFIT

Análisis y diseño de algoritmos

Documentación Práctica 1

Ismael García Ceballos

Carlos Alberto Álvarez Henao

Abril 05 de 2026

1. Introducción:

El presente informe documenta el desarrollo del Trabajo Practico 01 del curso Análisis y Diseño de Algoritmos de la Universidad EAFIT. El objetivo es implementar, analizar y comparar dos paradigmas fundamentales del diseño de algoritmos como lo son la Fuerza Bruta y el Backtracking, a través de dos problemas clásicos de la computación. El Ejercicio 1 aborda la generación de permutaciones con restricciones usando enumeración exhaustiva. El Ejercicio 2 resuelve el problema de coloración de grafos usando Backtracking con poda temprana, comparando su eficiencia contra la fuerza bruta. Ambos ejercicios incluyen implementación en C++17, análisis de complejidad temporal y espacial, experimentos de medición de tiempos y respuesta a las preguntas guía del enunciado.

2. Ejercicio 1 - Permutaciones con Restricciones:

Enfoque: Fuerza Bruta

2.1 Descripción del problema:

Dado un conjunto de n enteros positivos distintos, se desea generar todas las permutaciones posibles y retener aquellas que satisfacen la siguiente restricción para todo i en {0, 1, ..., n-2}:

```
P[i] <= 2 * P[i+1]
```

Es decir, ningún elemento puede ser mayor que el doble del elemento que le sigue en la permutación. El programa debe reportar el total de permutaciones generadas (exactamente n!), el total de permutaciones validas, y listarlas

2.2 Algoritmo y justificación del enfoque:

Se utiliza la función `next_permutation` de la STL de C++, que genera todas las permutaciones en orden lexicográfico a partir de un arreglo ordenado ascendentemente. Esta elección se justifica porque:

- Garantiza exactamente n! iteraciones cuando el arreglo esta ordenado inicialmente.
- Es eficiente en tiempo y espacio (O(n) por llamada, O(n) de memoria adicional).
- Permite una implementación limpia y directa del enfoque de fuerza bruta.

Este enfoque se clasifica como Fuerza Bruta porque la restricción se aplica después de generar cada permutación completa. No se descarta ninguna rama anticipadamente: el algoritmo explora el espacio completo de n! soluciones sin importar si una permutación parcial ya viola la restricción.

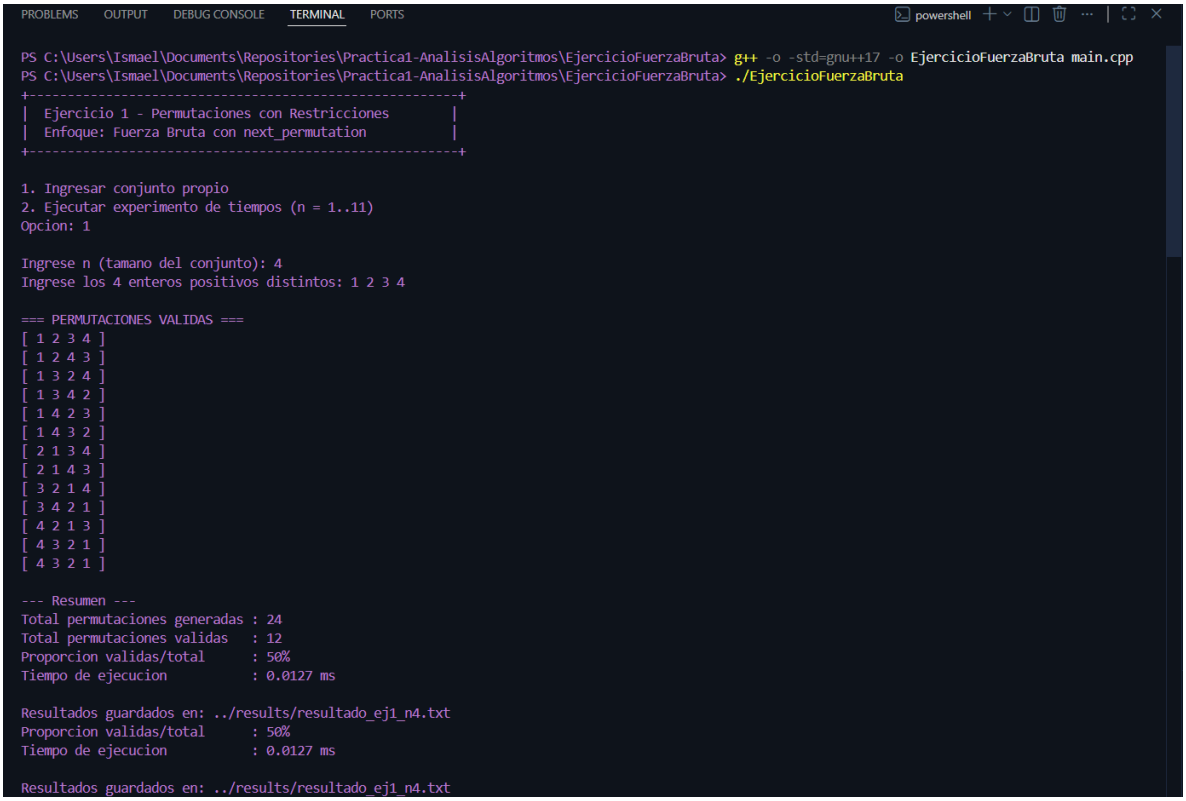
2.3 Pseudocódigo:

```
leer n
leer A[0..n-1]    // conjunto de enteros distintos
ordenar A asc    // garantiza comenzar desde la permutación
menor

total_generadas = 0
total_validas   = 0

hacer:
    total_generadas += 1
    valida = verdadero
    para i en [0..n-2]:
        si A[i] > 2 * A[i+1]:
            valida = falso
            break
    si valida:
        imprimir A
        total_validas += 1
mientras next_permutation(A) exista

imprimir "total_generadas =", total_generadas
imprimir "total_validas =", total_validas
```

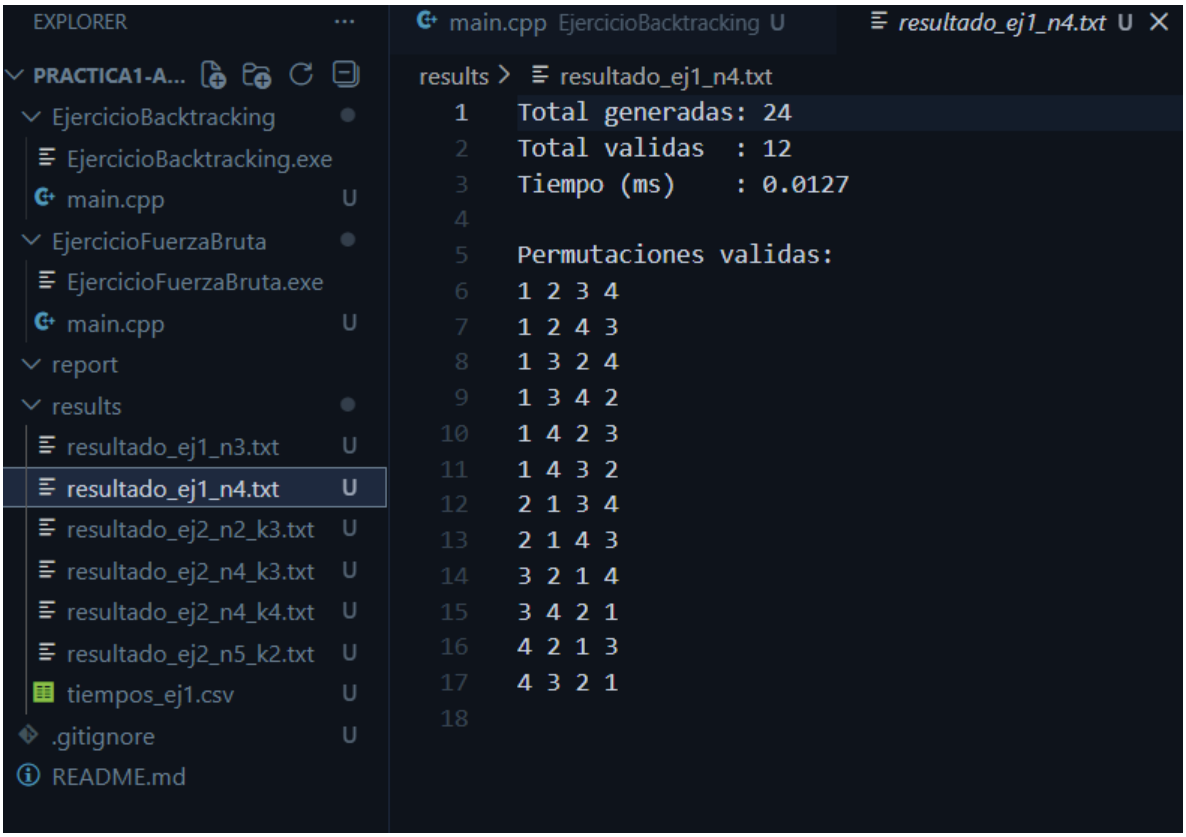
2.4 Ejecución:

Ejemplo: Para n = 4, A = 1, 2, 3, 4

Parámetro	Valor
Conjunto de entrada	{1, 2, 3, 4}
Total de permutaciones generadas	24 = 4!
Total de permutaciones válidas	12
Proporción válidas / total	50 %
Tiempo de ejecución	0.0127 ms



```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS                                   powershell

PS C:\Users\Ismael\Documents\Repositories\Practica1-AnalisisAlgoritmos\EjercicioFuerzaBruta> g++ -o -std=gnu++17 -o EjercicioFuerzaBruta main.cpp
PS C:\Users\Ismael\Documents\Repositories\Practica1-AnalisisAlgoritmos\EjercicioFuerzaBruta> ./EjercicioFuerzaBruta
+-----------------------------------------------------+
|  Ejercicio 1 - Permutaciones con Restricciones      |
|  Enfoque: Fuerza Bruta con next_permutation         |
+-----------------------------------------------------+

1. Ingresar conjunto propio
2. Ejecutar experimento de tiempos (n = 1..11)
Opcion: 1

Ingrese n (tamano del conjunto): 4
Ingrese los 4 enteros positivos distintos: 1 2 3 4

=== PERMUTACIONES VALIDAS ===
[ 1 2 3 4 ]
[ 1 2 4 3 ]
[ 1 3 2 4 ]
[ 1 3 4 2 ]
[ 1 4 2 3 ]
[ 1 4 3 2 ]
[ 2 1 3 4 ]
[ 2 1 4 3 ]
[ 3 2 1 4 ]
[ 3 4 2 1 ]
[ 4 2 1 3 ]
[ 4 3 2 1 ]
[ 4 3 2 1 ]

--- Resumen ---
Total permutaciones generadas : 24
Total permutaciones validas   : 12
Proporcion validas/total      : 50%
Tiempo de ejecucion           : 0.0127 ms

Resultados guardados en: ../results/resultado_ej1_n4.txt
Proporcion validas/total      : 50%
Tiempo de ejecucion           : 0.0127 ms

Resultados guardados en: ../results/resultado_ej1_n4.txt
```



2.5 Análisis de complejidad:

Temporal: O(n! * n)

Espacial: O(n)

La complejidad O(n! * n) es inherente al enfoque de fuerza bruta: no existe forma de reducirla sin cambiar de paradigma (por ejemplo, usando Backtracking con poda temprana). Para n >= 13 el tiempo de ejecución supera los varios minutos en hardware convencional.

2.6 Experimento de tiempos:

Se midió el tiempo de ejecución para n = 1 hasta 11 usando el conjunto A = {1, 2, ..., n}. Los tiempos fueron medidos con `std::chrono::high_resolution_clock`

```
PS C:\Users\Ismael\Documents\Repositories\Practica1-AnalisisAlgoritmos\EjercicioFuerzaBruta> ./EjercicioFuerzaBruta
+-------------------------------------------------------+
|  Ejercicio 1 - Permutaciones con Restricciones        |
|  Enfoque: Fuerza Bruta con next_permutation           |
+-------------------------------------------------------+

1. Ingresar conjunto propio
2. Ejecutar experimento de tiempos (n = 1..11)
Opcion: 2

+-----------------------------------------+
|   EXPERIMENTO DE TIEMPOS (n variable)    |
+-----------------------------------------+
n     n!            Validas         Tiempo (ms)
-------------------------------------------------
1     1             1               0.0002
2     2             2               0.0005
3     6             4               0.0008
4     24            12              0.0021
5     120           36              0.0118
6     720           144             0.0494
7     5040          576             0.3264
8     40320         2880            2.7719
9     362880        14400           24.0085
10    3628800       86400           209.459
11    39916800      518400          2168.49

Tabla de tiempos guardada en: ../results/tiempos_ej1.csv
PS C:\Users\Ismael\Documents\Repositories\Practica1-AnalisisAlgoritmos\EjercicioFuerzaBruta>
```

Conclusión: el tiempo de ejecución crece factorialmente. A partir de n = 12 (479,001,600 permutaciones) el tiempo estimado supera los 20 segundos, y para n = 13 superaría varios minutos. El umbral practico en este equipo es n = 11 (~2.1 s), siendo n = 12 el primer valor impracticable. Adicionalmente, se observa que la proporción de permutaciones validas respecto al total tiende a decrecer a medida que n crece: para n=4 es 50%, para n=5 es 30%, para n=10 es ~2.4%. Esto se debe a que la restricción P[i] <= 2*P[i+1] se vuelve cada vez más difícil de satisfacer en todas las posiciones simultáneamente.

2.7 Preguntas guía:

P1: ¿Que ocurre con la proporción de permutaciones validas respecto al total a medida que n crece?

La proporción decrece sostenidamente. Para n=4 es 50%; para n=5 es 30%; para n=10 es aproximadamente 2.4% (86,400 / 3,628,800). Esto se debe a que la restricción P[i] <= 2*P[i+1] debe satisfacerse en todos los n-1 pares consecutivos. A medida que crece n, la probabilidad de que una permutación aleatoria cumpla la condición en todas las

posiciones decrece exponencialmente, ya que cada par tiene una probabilidad menor de 1 de ser válido de forma independiente.

P2: Si se quisiera mejorar el enfoque con Backtracking, ¿en qué paso del algoritmo se podría construir el árbol de búsqueda?

El árbol de búsqueda se construiría colocando elementos en la permutación uno a uno, de la posición 0 a la n-1. En cada paso i, antes de colocar un elemento x en la posición i, se verificaría si el elemento ya colocado en la posición i-1 (llamémoslo prev) satisface prev <= 2*x. Si no, toda la sub-permutacion que empieza con esa elección es invalida y se poda sin explorarla. Esto evitaría generar y evaluar enormes ramas del árbol que de todas formas serian descartadas.

P3: ¿Por qué `next_permutation` garantiza exactamente n! permutaciones si el arreglo esta ordenado inicialmente?

`next_permutation` genera las permutaciones en orden lexicográfico estricto. Comenzando desde la permutación lexicográficamente mínima (arreglo ordenado ascendentemente), avanza en pasos mínimos hasta alcanzar la permutación máxima (orden descendente), visitando cada una de las n! permutaciones del conjunto exactamente una vez. Cuando alcanza la permutación máxima y se le solicita la siguiente, regresa a la permutación mínima y retorna false, indicando que el ciclo completo de n! permutaciones ha sido recorrido.

3. Ejercicio 2 - Coloración de Grafos con k Colores:

Enfoque: Backtracking con poda + verificación por Fuerza Bruta

3.1 Descripción del problema:

Dado un grafo no dirigido G = (V, E) con n vértices y m aristas, representado mediante una matriz de adyacencia, y un entero k de colores disponibles (numerados 1..k), se desea determinar si existe una k-coloración valida: una asignación de colores a los vértices tal que ningún par de vértices adyacentes comparta el mismo color.

El programa debe: encontrar la primera coloración valida (si existe), contar el total de k-coloraciones validas distintas, y comparar la eficiencia del Backtracking contra la fuerza bruta (kn combinaciones).

3.2 Algoritmo y justificación del enfoque:

El algoritmo asigna colores vértice a vértice (de 0 a n-1). Para cada vértice v prueba cada color c en {1..k} y verifica con `esSeguro(v, c)` si ningún vecino ya coloreado tiene el color c. Si es seguro, asigna color[v] = c y recursa al vértice v+1. Si ninguna asignación es válida, retrocede (backtrack) desasignando el color del vértice actual.

Algoritmo de Fuerza Bruta (verificación)

Para verificar la corrección del Backtracking, se implementa la fuerza bruta que enumera las kn combinaciones posibles usando representación en base k, y cuenta cuantas son coloraciones válidas. Ambos métodos deben arrojar el mismo conteo.

3.3 Pseudocódigo:

```
leer n, k
leer matriz_ady[0..n-1][0..n-1]    // 1 si hay arista, 0 si no

color[0..n-1] = 0    // 0 = sin color asignado
total_soluciones = 0
primera_guardada = falso

es_seguro(v, c):
    para cada u adyacente a v:
        si color[u] == c: retornar falso
    retornar verdadero

bt(v):
    si v == n:
        total_soluciones += 1
```

```
        si !primera_guardada:
            guardar color[]; primera_guardada = verdadero
        retornar
    para c en [1..k]:
        si es_seguro(v, c):
            color[v] = c
            bt(v + 1)
            color[v] = 0    // backtrack

bt(0)

si total_soluciones == 0:
    imprimir "No existe una", k, "-coloreacion valida"
si no:
    imprimir "Total de", k, "-coloraciones:", total_soluciones
    imprimir "Primera solucion:", primera_solucion
```

3.4 Ejecución:

Ejemplo 1: Ciclo C4 (n=4, k=3)

Grafo: ciclo de 4 vertices (0-1-2-3-0). Aristas: {(0,1),(1,2),(2,3),(0,3)}.

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS powershell

```
PS C:\Users\Ismael\Documents\Repositories\Practica1-AnalisisAlgoritmos\EjercicioBacktracking> ./EjercicioBacktracking
+-------------------------------------------------------+
|  Ejercicio 2 - Coloracion de Grafos con k Colores     |
|  Backtracking con poda + Fuerza Bruta                 |
+-------------------------------------------------------+

Opciones de entrada:
  1. Ingresar grafo manualmente
  2. Ejemplo 1 - Ciclo C4   (n=4, k=3)
  3. Ejemplo 2 - Completo K4 (n=4, k=4)
  4. Ejemplo 3 - Bipartito K2,3 (n=5, k=2)
Opcion: 2
[Ejemplo 1] Ciclo C4 (4 vertices, k=3)

Matriz de adyacencia:
  0 1 2 3
0 0 1 0 1
1 1 0 1 0
2 0 1 0 1
3 1 0 1 0

Primera solucion encontrada (Backtracking):
  color[0] = 1
  color[1] = 2
  color[2] = 1
  color[3] = 2

=== Comparacion de Enfoques ===
Metodo                    Coloraciones      Nodos explorados    Tiempo (ms)
--------------------------------------------------------------------------------
Backtracking (poda)       18                40                  0.0068
Fuerza Bruta (k^n)        18                81                  0.0139

[OK] Ambos enfoques coinciden: 18 coloraciones validas.
Reduccion de nodos por poda: 50.6%

Resultados guardados en: ../results/resultado_ej2_n4_k3.txt
```

EXPLORER ···

PRACTICA1-ANALISISALGORITMOS
- EjercicioBacktracking
  - EjercicioBacktracking.exe
  - main.cpp U
- EjercicioFuerzaBruta
  - EjercicioFuerzaBruta.exe
  - main.cpp U
- report
- results
  - resultado_ej1_n3.txt U
  - resultado_ej1_n4.txt U
  - resultado_ej2_n2_k3.txt U
  - resultado_ej2_n4_k3.txt U
  - resultado_ej2_n4_k4.txt U
  - resultado_ej2_n5_k2.txt U
  - tiempos_ej1.csv U
- .gitignore U
- README.md

main.cpp EjercicioBacktracking U | *resultado_ej2_n4_k3.txt* U

results > resultado_ej2_n4_k3.txt

```
=== Ejercicio 2 - Coloracion de Grafos ===
n=4  k=3

--- Backtracking ---
Coloraciones validas : 18
Nodos explorados     : 40
Tiempo (ms)          : 0.0068

--- Fuerza Bruta ---
Coloraciones validas : 18
Nodos explorados     : 81
Tiempo (ms)          : 0.0139

Primera solucion (BT):
  color[0] = 1
  color[1] = 2
  color[2] = 1
  color[3] = 2
```

Ejemplo 2: Grafo completo K4 (n=4, k=4)

Grafo: K4, todos los pares de vértices están conectados (m=6 aristas).

```
PS C:\Users\Ismael\Documents\Repositories\Practica1-AnalisisAlgoritmos\EjercicioBacktracking> ./EjercicioBacktracking
+-------------------------------------------------------+
|  Ejercicio 2 - Coloracion de Grafos con k Colores     |
|  Backtracking con poda + Fuerza Bruta                 |
+-------------------------------------------------------+

Opciones de entrada:
  1. Ingresar grafo manualmente
  2. Ejemplo 1 - Ciclo C4   (n=4, k=3)
  3. Ejemplo 2 - Completo K4 (n=4, k=4)
  4. Ejemplo 3 - Bipartito K2,3 (n=5, k=2)
Opcion: 3
[Ejemplo 2] Grafo completo K4 (4 vertices, k=4)

Matriz de adyacencia:
  0 1 2 3
0 0 1 1 1
1 1 0 1 1
2 1 1 0 1
3 1 1 1 0

Primera solucion encontrada (Backtracking):
  color[0] = 1
  color[1] = 2
  color[2] = 3
  color[3] = 4

=== Comparacion de Enfoques ===
Metodo                    Coloraciones      Nodos explorados    Tiempo (ms)
-------------------------------------------------------------------------------
Backtracking (poda)       24                65                  0.0107
Fuerza Bruta (k^n)        24                256                 0.0303

[OK] Ambos enfoques coinciden: 24 coloraciones validas.
Reduccion de nodos por poda: 74.6%

Resultados guardados en: ../results/resultado_ej2_n4_k4.txt
```

```
EXPLORER                                  main.cpp EjercicioBacktracking U    resultado_ej2_n4_k4.txt U
PRACTICA1-A...                            results > resultado_ej2_n4_k4.txt
  EjercicioBacktracking                   1  === Ejercicio 2 - Coloracion de Grafos ===
    EjercicioBacktracking.exe             2  n=4  k=4
    main.cpp                U             3
  EjercicioFuerzaBruta                    4  --- Backtracking ---
    EjercicioFuerzaBruta.exe              5  Coloraciones validas : 24
    main.cpp                U             6  Nodos explorados     : 65
  report                                  7  Tiempo (ms)          : 0.0107
  results                                 8
    resultado_ej1_n3.txt    U             9  --- Fuerza Bruta ---
    resultado_ej1_n4.txt    U             10 Coloraciones validas : 24
    resultado_ej2_n2_k3.txt U             11 Nodos explorados     : 256
    resultado_ej2_n4_k3.txt U             12 Tiempo (ms)          : 0.0303
    resultado_ej2_n4_k4.txt U             13
    resultado_ej2_n5_k2.txt U             14 Primera solucion (BT):
    tiempos_ej1.csv         U             15   color[0] = 1
  .gitignore                U             16   color[1] = 2
  README.md                               17   color[2] = 3
                                          18   color[3] = 4
                                          19
```

Ejemplo 3: Grafo bipartito K2,3 (n=5, k=2)

Grafo bipartito con dos grupos: {v0,v1} y {v2,v3,v4}, todos conectados entre grupos.

```
PS C:\Users\Ismael\Documents\Repositories\Practica1-AnalisisAlgoritmos\EjercicioBacktracking> ./EjercicioBacktracking
+-------------------------------------------------------+
|  Ejercicio 2 - Coloracion de Grafos con k Colores     |
|  Backtracking con poda + Fuerza Bruta                 |
+-------------------------------------------------------+

Opciones de entrada:
  1. Ingresar grafo manualmente
  2. Ejemplo 1 - Ciclo C4   (n=4, k=3)
  3. Ejemplo 2 - Completo K4 (n=4, k=4)
  4. Ejemplo 3 - Bipartito K2,3 (n=5, k=2)
Opcion: 4
[Ejemplo 3] Grafo bipartito K2,3 (5 vertices, k=2)

Matriz de adyacencia:
  0 1 2 3 4
0 0 0 1 1 1
1 0 0 1 1 1
2 1 1 0 0 0
3 1 1 0 0 0
4 1 1 0 0 0

Primera solucion encontrada (Backtracking):
  color[0] = 1
  color[1] = 1
  color[2] = 2
  color[3] = 2
  color[4] = 2

=== Comparacion de Enfoques ===
Metodo                      Coloraciones      Nodos explorados    Tiempo (ms)
--------------------------------------------------------------------------------
Backtracking (poda)         2                 13                  0.007
Fuerza Bruta (k^n)          2                 32                  0.0253

[OK] Ambos enfoques coinciden: 2 coloraciones validas.
Reduccion de nodos por poda: 59.4%

Resultados guardados en: ../results/resultado_ej2_n5_k2.txt
```

```
EXPLORER
PRACTICA1-A...
EjercicioBacktracking
  EjercicioBacktracking.exe
  main.cpp                U
EjercicioFuerzaBruta
  EjercicioFuerzaBruta.exe
  main.cpp                U
report
results
  resultado_ej1_n3.txt    U
  resultado_ej1_n4.txt    U
  resultado_ej2_n2_k3.txt U
  resultado_ej2_n4_k3.txt U
  resultado_ej2_n4_k4.txt U
  resultado_ej2_n5_k2.txt U
  tiempos_ej1.csv         U
.gitignore                U
README.md
```

main.cpp EjercicioBacktracking U | *resultado_ej2_n5_k2.txt* U

results > resultado_ej2_n5_k2.txt

```
=== Ejercicio 2 - Coloracion de Grafos ===
n=5  k=2

--- Backtracking ---
Coloraciones validas : 2
Nodos explorados     : 13
Tiempo (ms)          : 0.007

--- Fuerza Bruta ---
Coloraciones validas : 2
Nodos explorados     : 32
Tiempo (ms)          : 0.0253

Primera solucion (BT):
  color[0] = 1
  color[1] = 1
  color[2] = 2
  color[3] = 2
  color[4] = 2
```

3.5 Análisis de complejidad:

Backtracking: O(k^n * n) — peor caso:

En cada nivel de recursión (n niveles) se prueban hasta k colores, dando un arbol de kn hojas en el peor caso. La verificación `esSeguro(v,c)` cuesta O(n). En la práctica, la poda reduce drásticamente los nodos explorados, especialmente en grafos densos.

Fuerza Bruta: O(k^n * n^2):

Genera las kn combinaciones y para cada una verifica todos los pares de aristas (O(n2)), resultando en O(kn * n2). No existe mecanismo de poda.

3.6 Preguntas guía:

P1: ¿Cuantos nodos explora el Backtracking vs la Fuerza Bruta en el ejemplo del Ciclo C4 (k=3)?

Backtracking: 40 nodos. Fuerza Bruta: 81 nodos (= 3^4 = k^n). El Backtracking descarta ramas en cuanto detecta que el color propuesto para un vértice conflictúa con algún vecino ya coloreado, evitando explorar todas las extensiones de esa asignación parcial.

P2: ¿Qué tipo de grafo maximizaría el efecto de la poda? ¿Y cuál lo minimizaría?

Maximiza la poda: el grafo completo Kn (alta densidad, muchas aristas). Cada vértice nuevo tiene n-1 vecinos ya coloreados, lo que elimina inmediatamente la mayoría de opciones de color. En el ejemplo K4 con k=4 la poda elimina el 74.6% de los nodos. Minimiza la poda: el grafo vacío (sin aristas). `esSeguro` siempre retorna true, ningún color genera conflicto, el Backtracking degenera completamente en fuerza bruta y explora los k^n nodos sin poda alguna.

P3: ¿Qué cambio se debe hacer al algoritmo si solo se quiere saber si existe al menos una k-coloración, sin contar todas?

Agregar una variable booleana 'encontrado'. En cuanto se alcanza v == n (primera solución completa), se asigna encontrado = true y se retorna inmediatamente en todos

los niveles de la recursión mediante una condición de salida anticipada. Esto transforma la búsqueda exhaustiva en una búsqueda de primera solución (DFS con early exit), que en grafos con muchas soluciones puede ser drásticamente más rápida.

P4: ¿Que relación existe entre este problema y el Numero Cromático del grafo?

El numero cromático chi(G) es el mínimo k para el cual existe una k-coloración valida de G. Este algoritmo resuelve el problema de decision: dado k, ¿existe una k-coloración? Ejecutándolo para k = 1, 2, 3, ... y deteniendo al primer k con solución, se obtiene chi(G). Resultados clásicos: chi(árbol) = 2, chi(ciclo par) = 2, chi(ciclo impar) = 3, chi(Kn) = n. Calcular chi(G) es un problema NP-difícil en general

4. Conclusiones:

Fuerza Bruta vs Backtracking:

La fuerza bruta garantiza encontrar todas las soluciones, pero su costo crece de forma factorial/exponencial con el tamaño de la entrada, haciéndola impracticable rápidamente. El Backtracking, mediante la poda temprana, reduce el espacio de búsqueda explotando las restricciones del problema, con reducciones de nodos observadas entre el 50% y el 75% en los grafos evaluados.

Limite practico de la Fuerza Bruta en permutaciones:

Para el problema de permutaciones con restricciones, el umbral practico en el equipo de prueba es n = 11 (~2.1 segundos). A partir de n = 12 el tiempo supera los 20 segundos y crece de forma factorial, confirmando la complejidad O(n! * n).

Efectividad de la poda en grafos densos:

El efecto de la poda del Backtracking es directamente proporcional a la densidad del grafo. Los grafos completos (alta densidad) maximizan la poda; los grafos vacíos la anulan por completo. Esto confirma que el Backtracking es más efectivo cuando las restricciones del problema son fuertes y frecuentes.

Verificación cruzada:

La implementación dual (Backtracking + fuerza bruta) en el Ejercicio 2 permitió verificar que ambos enfoques arrojan exactamente el mismo número de coloraciones validas en todos los casos probados, validando la corrección del algoritmo de Backtracking implementado.

5. Uso de Herramientas de Inteligencia Artificial:

Claude AI se utilizó como herramienta de desarrollo de las siguientes maneras específicas:

1. Comentarios en el código: Claude generó los comentarios en línea que explican la función de cada función, bloque y línea clave en ambos archivos `main.cpp`.

2. Claridad de la salida en consola: Claude mejoró el formato de las secciones que imprimen os resultados de la ejecución (tablas resumen, encabezados y mensajes de resultado) para que la salida del programa sea más fácil de leer e interpretar.

3. Estructura de doble enfoque (Ejercicio 2): Claude sugirió e implementó la arquitectura de combinar Backtracking y Fuerza Bruta en un solo programa (`EjercicioBacktracking/main.cpp`), incluyendo la tabla comparativa de nodos explorados, colores válidos y tiempo de ejecución.

6. Referencias:

https://claude.ai/

https://www.geeksforgeeks.org/dsa/introduction-to-backtracking-2/

https://brilliant.org/wiki/recursive-backtracking/

https://pencilprogrammer.com/algorithms/graph-coloring-problem/

https://brilliant.org/wiki/graph-coloring-and-chromatic-numbers/

https://www.theochem.ru.nl/~pwormer/Knowino/knowino.org/w/index5de9.html?title=Graph_coloring&printable=yes